Functional Requirements:

1. User Management (Authentication & Authorization)

- Users should be able to register and log in using JWT-based authentication.
- Admin users should have additional privileges like managing orders, users, and inventory.
- Regular users should only be able to view products, place orders, and view their order history.

2. Product Management

- Admin should be able to:
  - Add, update, or delete products.
  - Assign products to categories (e.g., electronics, clothing).
  - Track inventory (stock count).
- Users should be able to:
  - View products and product details (name, description, price, category, and availability).
  - Search products by name, category, and price.

3. Order Management

- Users can place an order containing one or more products.
  - When an order is placed, it should reduce the available stock of the products.
  - Each order should have a status (Pending, Shipped, Delivered, Cancelled).
  - Orders should be linked to a user and must be recorded with details like product(s), shipping address, and payment method.

4. Shopping Cart

- Users should be able to add products to a shopping cart, review the cart, and place an order.
- The cart should store products with their quantities and prices.

5. Admin Dashboard

- The admin should be able to:
  - View a list of all orders (filtered by status, date, etc.).
  - View user information and order history.
  - Monitor inventory and adjust stock levels manually.

6. Payment Gateway Simulation

- Simulate the integration of a payment gateway (like PayPal or Stripe).
- For simplicity, just create a mock service that processes payments and returns a success/failure status based on some logic.

7. Logging and Error Handling

- Implement structured logging (preferably with **Serilog** or **NLog**) to log API requests, errors, and important operations.
- Return appropriate HTTP status codes and messages in the response (e.g., 404 for "Not Found", 400 for validation errors, etc.).

8. Unit Testing & Integration Testing

- Write unit tests to verify business logic.
- Write integration tests for your API endpoints.

9. Asynchronous Operations

- Handle long-running processes asynchronously (e.g., sending confirmation emails after an order is placed).
- Implement background jobs using **Hangfire** or **BackgroundService** in .NET Core for handling tasks like email notifications or inventory updates.

Technical Requirements:

1. **.NET Core Version**: Use **.NET 6 or higher**.
2. **Database**: Use **Entity Framework Core** for database operations (preferably with a relational database like SQL Server or PostgreSQL).
3. **Authentication**: Use **JWT (JSON Web Tokens)** for securing API endpoints.
4. **API Design**:
   - Follow **RESTful principles** to design endpoints.
   - Use **Swagger** (OpenAPI) for API documentation.
   - Use proper HTTP methods (GET, POST, PUT, DELETE).
5. **Testing Frameworks**:
   - **xUnit** for unit tests.
   - **Moq** for mocking dependencies.
   - **TestServer** for integration testing.
6. **API Rate Limiting**: Optionally, implement basic rate limiting for public API endpoints.
7. **API Versioning**: Use **API versioning** (e.g., `v1`, `v2`).
8. **Caching**: Use **Redis** for caching product data to improve performance.

Additional Features (Optional Enhancements):

1. **Product Recommendation System**:
   - Implement a recommendation engine using basic algorithms (e.g., most popular products, or products similar to the ones the user has viewed).
2. **Search and Filter**:
   - Allow users to filter products by attributes like price range, brand, or rating.

Evaluation Criteria:

1. **Code Quality**:
    - Adherence to **SOLID principles**.
    - Proper use of design patterns (e.g., repository pattern, unit of work).
    - Use of asynchronous programming for non-blocking tasks.
    - Proper exception handling and logging.
2. **API Design**:
    - Clear, consistent, and clean API design.
    - Proper HTTP status codes, request/response structure.
    - Well-documented API with Swagger.
3. **Testing**:
    - Comprehensive unit and integration tests.
    - Mocking of external dependencies like payment gateways and email services.
4. **Performance & Scalability**:
    - Consideration of potential scalability concerns, such as caching and efficient database queries.
5. **UI/UX (Optional)**:
    - If the candidate chooses to implement a front-end (like React, Angular, or Blazor), evaluate the user experience and responsiveness.